Optimizing parameters of robotic task-oriented programming via a multiphysics simulation

1st Michele Delledonne
Department of Mechanical and Industrial Engineering
University of Brescia
via Branze 38, 25123 Brescia, Italy
https://orcid.org/0000-0001-5236-2706

2nd Enrico Villagrossi
Institute of Intelligent Industrial Technologies and Systems for Advanced Manufacturing,
National Research Council of Italy
via A. Corti 12, 20133 Milan, Italy
https://orcid.org/0000-0002-9493-4175

3rd Manuel Beschi
Department of Mechanical and Industrial Engineering
University of Brescia
via Branze 38, 25123 Brescia, Italy
https://orcid.org/0000-0002-8845-2313

***Abstract*—The programming complexity of industrial robots significantly limits their expansion in complex industrial applications. Consequently, research has focused extensively on the development of intuitive programming methods.**

This article proposes a framework for task-oriented programming introducing an intuitive and modular task structure. The framework provides an algorithm able to optimize the execution parameter of the tasks. A physical simulation environment allows accurate parameter optimization in a virtual environment providing feasible and safe results. Efficiency tests demonstrated the method's effectiveness, and a comparison with genetic and Bayesian -based ones have been conducted.

***Index Terms*—Robotic tasks optimization, Task-oriented programming, Intuitive robot programming**

I. Introduction

The field of robotics has witnessed significant advancements in recent years, revolutionizing industries and automation processes [1]. However, this technological improvement is not accompanied by an equal improvement in programming techniques. Some of the prominent programming methods used today include:

- Drive-through programming: manually guiding the robot through desired motions using the robot Teach Pendant (TP) or a handheld device. The operator records robot waypoints replicated by the robot during task execution.
- Robot-oriented programming languages: industrial robots utilize specialized programming languages such as Robotic Programming Language (RPL), KUKA Robot Language (KRL), or ABB's Rapid Programming Language. These languages provide a syntax and set of commands that allow programmers to specify robot motions, control logic, and interact with peripherals. Programming languages provide flexibility and fine-grained control but require specific language and robot system expertise.
- Offline programming: robot programming is performed offline on a separate computer. Offline programming software allows operators to virtually create and simulate robot programs, using 3D models of the robot and the work environment. The generated program can then be transferred to the robot system for execution. Offline programming reduces downtime and enables programmers to optimize programs without interrupting production, but it requires an accurate environment model.

These programming methods often demand extensive knowledge of programming languages and robotics, limiting their accessibility to highly skilled individuals. To address this issue, the research world has proposed innovative programming methods as:

- Graphical programming interfaces: visually represent robot tasks and logic, allowing operators to drag and drop pre-defined blocks or symbols to create robot programs avoiding code writing [2]. Blockly and Scratch [3] are examples of visual programming languages that simplify the process using graphical blocks to represent programming instructions.
- Natural Language Processing (NLP): techniques enabling operators to interact with robots using natural language commands [4]. By leveraging speech recognition and language understanding algorithms, robots can interpret verbal instructions from the operator and convert them into executable actions [5].
- Virtual Reality (VR) programming: VR-based programming environments offer immersive and intuitive experiences for robot programming. Operators can visualize and manipulate virtual robot models and objects within a virtual environment [6]. Operators can use hand gestures and controllers to define robot motions, interact with virtual objects, and program complex tasks. VR programming provides an intuitive and immersive way to program robots, enabling operators to "step into" the virtual environment and manipulate robot behavior.
- Demonstration-based programming: it allows operators to teach robots tasks by physically demonstrating the desired actions [7]. Robots can learn and replicate the demonstrated actions through sensors and motion capture technologies [8].

These methods, while striving to simplify the programming process, may have specific limitations that make them less suitable for the industry world in some cases as:

- Low precision: intuitive programming approaches may prioritize simplicity over fine-grained control, leading to

reduced precision in specific industrial applications [9] [10]. Industries often require high precision and accuracy in robot movements, especially in assembly, welding, or inspection tasks.

- Difficulty in industrial environments: real environments can be complex, with various factors such as noise, vibrations, and unpredictable conditions. Intuitive programming approaches may not adequately address these challenges, making it difficult to program robots to operate reliably in such environments. The need to consider factors like collision avoidance, safety protocols, and interaction with other machinery further complicates the programming process [11]. For example, verbal programming is complicated in a noisy environment.
- Complex task definition: industrial tasks often involve intricate sequences of actions, complex decision-making, and interaction with multiple objects or machines [12]. Translating the complexity of real-world industrial scenarios into intuitive representations may result in oversimplification or inadequate task definition, limiting the applicability of intuitive programming methods.
- Complex integration in existing systems: adopting and integrating new programming methods into existing industrial systems can require significant time and effort. Intuitive programming approaches often require changes in workflow, training of operators, and potential reconfiguration of existing processes. This implementation time and associated costs can be perceived as barriers to their adoption in fast-paced industrial environments.

This paper presents the Task Optimization Framework (TOF) designed to enhance and support intuitive programming in robotics. TOF operates within a modular task definition structure, allowing for the creation of various and complex tasks. However, this modularity leads to the need for a detailed definition of parameters assigned to each module, which could result in slow application commissioning due to tuning time. Thus, TOF incorporates optimization algorithms that determine the best parameters set starting from standard values or values easily defined by the user using simulation and real-world data. Applying this procedure in a real environment is not recommended to ensure safe and problem-free commissioning. Therefore, TOF provides a physics simulation environment that faithfully reproduces the intended task. The Manipulation Framework (MF) proposed in [13] allow task-oriented programming. The MF allow to combine multiple actions to implement a task easily. The limitation lies in the rigidity of the actions predefined by the developers. The ability to generate new actions and the tuning of their parameters is addressed in this paper.

The paper is organized as follows: Section II provides a detailed description of TOF, focusing on the task structure, the parameter optimization algorithms, and the simulation environment. Section III presents the results and the discussion of tests to compare the optimization algorithms presented in this article with well-known algorithms. Section IV reports the conclusions and future works.

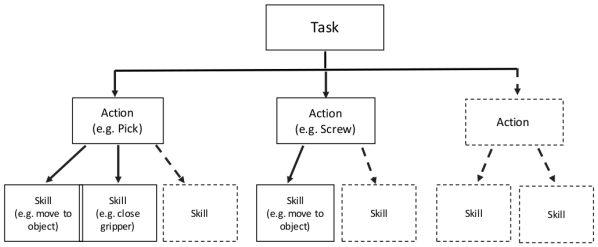


Fig. 1. *Task structure. Image taken from [13].*

II. Method

TOF aims to provide the ability to define and autonomously refine a task. This ability allows an operator to program a robot by providing only the essential and easily definable information without dealing with robot programming, such as program structure definition, waypoints teaching, etc..

TOF is built upon a flexible, intuitive, and easily implementable task definition. A modular task structure has been designed to achieve this, where knowledge of the basic components is sufficient to create any task. The basic component parameters and their logical order can be modified, allowing for various actions. To accomplish this, TOF relies on advanced software such as Behavior Trees (BT) and the Robot Operating System (ROS) [14] programming environment.

This task structure does not eliminate the necessity of defining all the basic component parameters. However, TOF aims to offer algorithms for optimizing parameter sets to alleviate this burden for the operators. Because these algorithms require many tests to achieve satisfactory results, conducting these tests in a real environment can be impractical. Therefore, provides a simulation environment for initial task optimization.

A. Task structure

Given the definitions in [13], [15] the *skills* represents primitive functions that the robot can perform, such as detecting surfaces through sensitive touch, simple movement, and gripping objects. These *skills* can be combined to create *actions*. This modular *actions* allows users to program *actions* using a block-based programming approach, which is intuitive and user-friendly. Ultimately, a sequence of *actions* forms a *task*. The *tasks* themselves are modular and can be easily programmed by the user.

Figure 1 shows the task structure. The idea is to provide a predefined set of actions to define a task quickly, but no restriction prevents a user from defining custom action. By exclusively utilizing these *actions*, programming shifts from a robot-oriented approach (focused on elementary robot actions) to a task-oriented one (emphasizing the entire task).

B. Behavior Tree

The structure of the task is defined using BTs. A BT is a mathematical model of plan execution. Its power is the capability to define a complex task combining simple

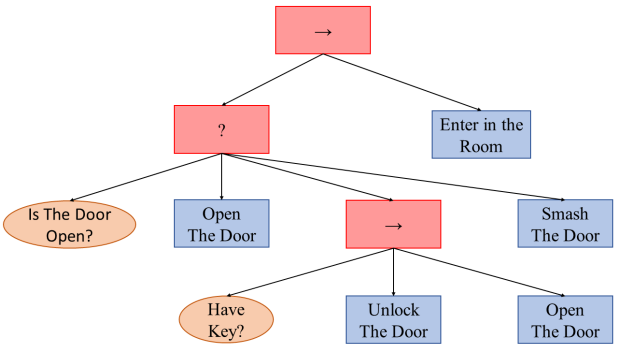


Fig. 2. Behavior tree example. Symbol "→": Sequence. Symbol "?": Fallback. Yellow oval: Decorator Node. Blue box: Leaf Nodes.

nodes; they are structured in a tree-like structure that is very understandable and user-friendly. Nodes can be of different types:

- Root Node: it serves as the starting point for the BT. It oversees the overall execution of the tree and manages the flow of behavior execution.
- Composite Nodes: nodes establish the structure and organization of behaviors within the tree. For instance, the Sequence node executes its child nodes sequentially until all are completed or one fails. The Fallback node executes its child nodes until one succeeds or all fail. Additionally, the Parallel node executes multiple nodes simultaneously.
- Decorator Nodes: they are responsible for altering the behavior of other nodes in the tree. They incorporate conditions, control execution frequency, or handle specific events within the BT structure.
- Leaf Nodes: they correspond to the concrete actions the robot needs to perform; in our case, they represent the *skills*. They reside at the tree's lowest level and have no child nodes. These nodes directly execute specific actions without further branching.

Figure 2 shows a simple example of a behavior tree.

TOF utilizes ROS controllers, which serve as the leaf nodes of the behavior tree. These controllers leverage ROS parameters to execute the skills, allowing for straightforward modification of the skills' execution. The task optimization process optimizes these parameters, enabling efficient and effective execution.

C. Task parameters optimization

The optimization of the task has been conceived as a trial and error process, starting from a set of parameters that are partly predefined and partly determined by the operator. The operator is responsible for defining the specific *action* types that the robot must perform, including any necessary positions for tasks such as grasping and releasing objects. The choice of the *action* type is essential in determining the goodness of task execution, and various criteria are used to evaluate different *action* types. For example, in the case of a peg-in-hole task, the primary evaluation focuses on the object's final position, precisely whether it is successfully placed in the hole, otherwise by its proximity to the hole. Secondary considerations include the task's completion time and the interaction forces exerted during the process. The availability of position information is essential for establishing a starting point in the optimization process, reducing optimization time. Initiating the optimization with the entire robot workspace significantly increases the complexity of the process.

The proposed algorithm follows a straightforward logic. It searches within the parameter space, starting from an initial set and updating the parameters only if the new values perform better than the previous ones. Parameter modifications are carried out individually and are influenced by the probabilities assigned to three possible variations: increasing, decreasing, or maintaining the parameter value. The probability variables are adjusted based on whether the new execution results. If the result improves, the probability increases, while it decreases otherwise. This variation encourages the algorithm to enhance the result relative to the specific variation. Evaluation indices are task-dependant. Typically, these indices include time, exchanged forces, and distances.

The outcome of execution is defined through a reward, which represents the quality of task execution. The metrics mentioned above are used to define the reward. Each metric is assigned a weight corresponding to its importance. The reward is obtained by multiplying each metric by its weight and then summing all the obtained values.

The weights are structured to establish a hierarchy among the indices. They create a significant distinction between the various indices, defining which ones should hold greater precedence over others or if they should be considered comparable. This distinction is achieved by setting the weights as multiples of 10. For instance, the least important index is assigned a weight of 1, the subsequent one is assigned a weight of 10, and so on. If a higher level of differentiation is desired, one can directly move to the next set of multiples. This approach allows for defining the indices that are prioritized for improvement. Equation (1) shows how the reward is calculated

$$R = \sum_{k=0}^{n} I_{e_k} w_{I_{e_k}} \tag{1}$$

where the evaluation index I_{e_k} indicates the $k\text{-}th$ selected indices used for assessing the task, while the $k\text{-}th$ index weight $w_{I_{e_k}}$ represents the weight assigned to that particular index. As mentioned, three possible variations can be taken on each parameter: increasing, decreasing, and unchanged. The decision of which variation to choose depends on the probability variables. For each parameter, three variables are associated, one for each variation. These variables can have values between 0 and 1; initially, all are set to 1. Equation (2) illustrates how these variables are modified after the execution of a task, namely, the probability is the average of all the past outcomes (1 success, 0 failure). R represents the reward obtained from the current execution, while R_o represents the best previous reward. The variation v is applied to a specific parameter, whether an increment, decrement, or maintaining

its value. This action is executed before the test that will yield the reward R. $P()$ represents the probability variable associated with v, influencing the choice of future actions. The variable k indicates the iteration we are working on, representing the number of variations v. Only the variable associated with executed action for each parameter will be modified, while the other probability variables will be kept constant.

$$\begin{cases} P(v)|_k = P(v)|_{k-1} \cdot \left(\frac{k-1}{k}\right) + \frac{1}{k} & \text{if } R > R_o \\ P(v)|_k = P(v)|_{k-1} \cdot \left(\frac{k-1}{k}\right) & \text{if } R \leq R_o \end{cases} \tag{2}$$

where $k = 1, ..., n$. Equation (3) reports how to select the variation v

$$\begin{cases} v = v_i & \text{if } r_v < P(v_i) \\ v = v_{eq} & \text{if } P(v_i) < r_v < (P(v_{eq}) + P(v_i)) \\ v = v_d & \text{if } r_v > P(v_{eq}) + P(v_i) \end{cases} \tag{3}$$

where

$$r_v = rand(0, P_s) \ , \tag{4}$$

represents a random value ranging from 0 to the sum of the probability variables

$$P_s = P(v_i) + P(v_{eq}) + P(v_d) \tag{5}$$

where $P(v_i)$, $P(v_{eq})$, and $P(v_d)$ represent, respectively, the probability variables associated with the incremental variation v_i, the decremental variation v_d, and the unchanged variation v_{eq} associated with the parameter under evaluation. This approach determines the likelihood of selecting a variation by comparing the variable's value to the values of the others.

Equation (6) describes how parameters are modified based on the chosen variation. p_k represents the parameter used in the subsequent test at the k-th iteration, while p_b represents the parameter given by the best reward. δ_{max} represents the maximum allowable variation, set to a value large enough to produce significant changes in the task but not so large as to deviate excessively from the objective. $rand$[0:1] samples from a uniform distribution a value between 0 and 1, introducing randomness into the variation value. This approach aims to explore the parameter space better

$$\begin{cases} p_k = p_b + \delta_{max} \ rand[0{:}1] & \text{if } v = v_i \\ p_k = p_b - \delta_{max} \ rand[0{:}1] & \text{if } v = v_d \\ p_k = p_b & \text{if } v = v_{eq} \end{cases} \tag{6}$$

Equation (6) is modified into (7) to weigh the modification depending on the probability variable associated with the chosen action:

$$\begin{cases} p_k = p_b + \delta_{max} \ P(v) \ rand \ [0{:}1] & \text{if } v = v_i \\ p_k = p_b - \delta_{max} \ P(v) \ rand \ [0{:}1] & \text{if } v = v_d \\ p_k = p_b & \text{if } v = v_{eq} \end{cases} \tag{7}$$

A second improvement affects the variation of the probability variable. In Equation (2), all the variations contribute equally, causing the influence of a future variation to diminish as the number of tests increases. However, the last variation should have the same influence as the historical data. In (8), the weights a and b can be set *a-priori* to determine the importance of the last parameter variation, with the constraint $0 < a < b$.

$$\begin{cases} P(v)|_k = P(v)|_{k-1} \cdot \left(\frac{a}{b}\right) + \frac{1}{b} & \text{if } R > R_o \\ P(v)|_k = P(v)|_{k-1} \cdot \left(\frac{a}{b}\right) & \text{if } R \leq R_o \end{cases} \tag{8}$$

D. Simulation environment

Parameter optimization requires an experimental campaign. Conducting real-world experiments can pose several problems, especially considering that initial tests often have the highest execution uncertainties. For this reason, it was deemed essential to utilize a dynamic simulation environment to perform all necessary tests before transferring the task to a real-world setting and proceeding with the final refinements. PyBullet was employed for this purpose, as it is a highly versatile and user-friendly physics engine and simulation library widely utilized in robotics, physics-based simulations, and reinforcement learning [16]. It supports various rigid body dynamics (open and closed chains), collision detection, and constraint solving, enabling users to create complex and realistic simulations. Moreover, PyBullet excels in handling multi-body dynamics, making it suitable for modeling articulated structures like robotic systems. It provides intuitive methods to define joints and constraints between bodies, enabling the simulation of intricate mechanical systems. By considering factors such as collisions, friction, and gravity, PyBullet enables the creation of comprehensive simulations.

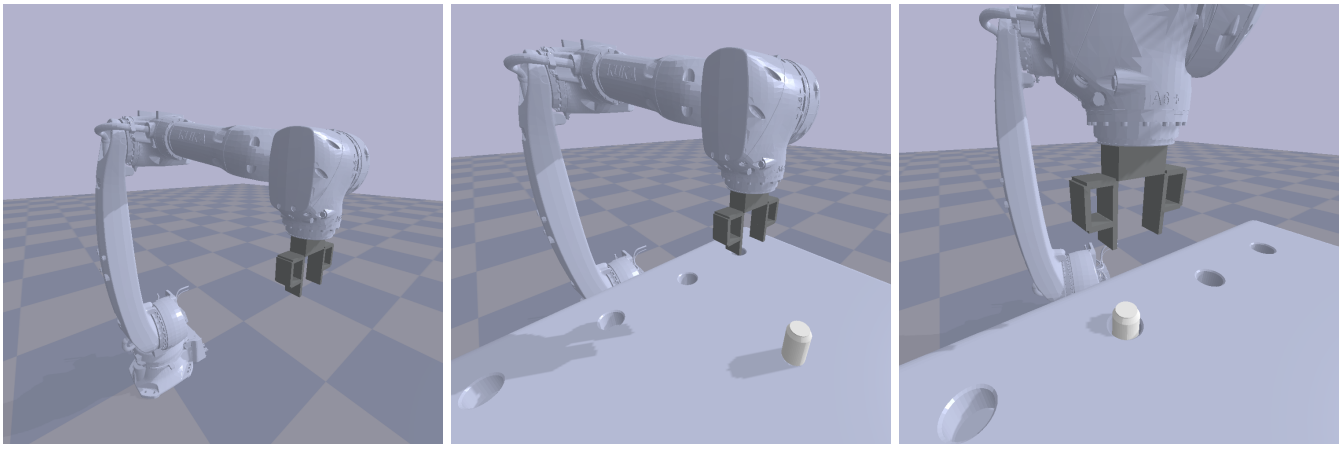
Figure 3(a) depicts the environment used in the simulation. The robot used is a KUKA KR 50 R2500 with a two-finger gripper as an end-effector. A force sensor has been installed between the robot flange and the gripper to measure the forces exchanged with the environment.

III. Results and Discussion

The optimization method proposed in this work has been compared to two well-known optimization algorithms. The genetic algorithm provided by PyGAD [17] and the Bayesian algorithm provided by Bayesian optimization [18] were used as benchmarks.

A. Optimization task

The optimization task is a peg-in-hole problem: the robot has to pick a can and insert it inside a hole. Figure 3(b) shows the simulation environment, the can to be grasped, and the holes where the can must be inserted. The depicted situation also serves as the starting state for our experiments. Figure 3(c) shows the desired final condition where the can should be located. A vision system in real environments should detect objects, providing the object position with uncertainties. Therefore, the algorithm needs to cope with these uncertainties. Thus, a misalignment of a few centimeters in the



(a) (b) (c)

Fig. 3. Simulation environment in PyBullet.

simulation scenario has been introduced between the actual and the estimated positions.

The following indices of the task need to be optimized:

- Goal achievement: in the final state of the task, the can should be in contact with the bottom of the hole. The robot may reach this state by simply dropping the can vertically from above the hole, but this should not be considered as the goal achieved. Therefore, the goal is marked as achieved when the can touches the bottom of the hole while still being gripped by the gripper. The minimum threshold between the can and the bottom of the hole is defined.
- Execution speed: the faster the task is completed, the better. One might think sending the robot at maximum speed and acceleration would be sufficient. However, this approach may lead to undesired situations, considering that the gripped object could be lost if the movements have excessively high accelerations. It might be tempting to grip the object with highly high force, but this could lead to its breakage. Therefore, the optimizer needs to find an effective trade-off.
- Interaction forces with the environment: the less force exchanged with the environment, the better. This index is as a safety condition since weaker interaction forces entail fewer risks.

Table I shows the skills parameters on which the optimization algorithms will have to work to optimize the task.

TABLE I

PARAMETERS TO OPTIMIZE. THE LAST COLUMN SHOWS THE PARAMETERS ON WHICH THE OPTIMIZATION ALGORITHMS WILL HAVE TO WORK.

Actions	Skills	Parameters
can pick	open gripper	torque
	move to can	relative position to can
		acceleration
		velocity
	close gripper	torque
	leave the table	velocity
can place	move to hole	relative position to hole
		acceleration
		velocity
	can insertion	max force
		velocity

B. Task reward

The reward assigned to the executions is calculated using the following indices: failure, final distance, duration, and maximum force exerted by the gripper, weighted in a hierarchical order. The failure index determines whether the task was successful or not. It has the highest weight, forcing the optimizer to focus on feasible solutions. The final distance refers to the distance between the can and the goal at the end of the execution while it is still gripped. This index holds the second highest weight because the final distance strongly indicates how close the execution is to achieving success. The duration is simply the time elapsed between the start and the end of an execution. It is assigned the third level of importance since the speed of task completion is significant but less crucial than executing it correctly. The maximum force exerted by the gripper represents the highest force detected between the gripper and the robot's flange. It serves as an indication of the forces exchanged with the environment. This index carries the lowest weight as interactions between the environment and the robot are expected to be minimal in this task. All weights assigned to each index are negative since a higher index value indicates a worse task performance. Therefore, the reward can be seen as a cost function to reduce its absolute value. Table II shows the weights and units of measurement relating to the indices seen. Considering Equation (1), the variables I_{e_k} represent the measurement of the indexes using their specified unit and $w_{I_{e_k}}$ represent their corresponding weights.

As mentioned earlier, object position errors were introduced during the tests, inevitably leading to a failed task in the initial execution. Therefore, the experiments primarily focus on the algorithm's ability to modify parameters to ensure correct task execution.

TABLE II
EVALUATION INDEXES. FAILURE: TASK SUCCESS NOT ACHIEVED; FINAL DISTANCE: DISTANCE BETWEEN HOLE BOTTOM AND THE CAN; DURATION: TASK DURATION; MAXIMUM FORCE: MAXIMUM FORCE EXCHANGED WITH THE ENVIRONMENT.

Index	Measure unit	Weight
Failure	boolean	-1000000
Final distance	mm	-1000
Duration	s	-100
Maximum force	N	-0.01

Thanks to the nature of the reward, it is possible to discretize situations when the can is not centered in the hole, when it is centered but not pushed to the bottom to complete the task successfully, and when the goal is accomplished. This discretization allows us to determine the number of iterations an algorithm takes to center the can and the number of iterations required to complete the task successfully. Taking this into account, we have decided to define the goodness of an algorithm based on the number of attempts made for centering the can and the number of attempts to achieve task success.

C. Tests result

The executed tests consist of running 20 optimization processes for each algorithm type. Each process includes the execution of 500 trials. The results consist of the average number of iterations required to achieve centering of the object and the number of iterations needed to complete the task successfully.

Table III displays the average iteration values for each algorithm type. Alg. 1 represents the algorithm that uses the basic exploration and modification of probability parameters (namely, (2) and (6)). Alg. 2 is an algorithm that incorporates exploratory behavior by integrating the probability variable while the modification of these variables remains as in the baseline (namely, (7) and (2)). Alg. 3 is the inverse of Alg. 2, meaning it uses classical exploration and modifies the probability variables with a more significant impact on the final execution ((6) and (8)). Alg. 4 involves exploration influenced by the probability variables and their weighted variation in the last execution (namely, (7) and (8)). For these tests, the coefficients in (8) are set equal to $a = 9$ and $b = 10$, respectively, to assign significant weight to the last execution.

The proposed algorithms are compared to Bayesian and Genetic algorithms. Bayesian algorithms are characterized by the number of initial points, the iteration number, and the number of random states. The initial point is the set of variable values to be optimized when the algorithm starts. More starting points indicate that the algorithm restarts from the beginning during the set of iterations. The iteration number indicates the number of tests that it will run. The number of random states indicates the number of starting points that should be calculated randomly. Different sets of parameters were defined to prevent the parameter selection from negatively impacting the experiments. The number of initial points takes on values of [3, 4, 5], the iteration number is set to 500, and the number of random states ranges from [1, 2, 3, 4, 5]. The combinations of these values define the parameter set for each optimization process. The function to optimize is the one that takes the task parameters as input and, after executing the task, provides the reward.

Genetic algorithms are optimization techniques that draw inspiration from natural selection and evolution. The underlying mechanism of genetic algorithms mirrors the principles of biological evolution. For this algorithm, the principal characterizing parameters are the generation size, the number of generations, the number of mating parents, the kept parents, and the percentage of genes to be mutated. The generation size represents the number of parameter sets in each population. The number of generations determines how often a new population is obtained using the previous population and its results. The number of parents to mate indicates how many old parameter sets should be used to generate new ones. The "kept" parents indicate how many members of the old sets should remain unchanged. For our experiments, the parameters have the following values: generation size of 10, number of generations of 50, number of mating parents [2, 3, 4], kept parents [-1, 0, 1, 2] (-1 indicates that all mating parents are kept), and a mutation percentage of genes of 10. The function to optimize corresponds to the one used in the genetic algorithm.

TABLE III
TESTS RESULT. CENTERING: ITERATIONS TO OBTAIN CAN CENTERING. SUCCESS: ITERATIONS TO OBTAIN A PERFECT EXECUTION (500 IS THE MAXIMUM ITERATIONS NUMBER IN THE SIMULATION).

	Bayesian	Genetic	Alg. 1	Alg. 2	Alg. 3	Alg. 4
Centering	305	400	56	73	93	48
Success	500*	489	156	118	319	145

The results reported in Table III reveal a clear distinction between the algorithms proposed in this article and those used for comparison. A possible explanation of the genetic-algorithm inability to adapt well to this type of optimization is the vast search space and the numerous parameters to optimize. Consequently, the algorithm requires many iterations to achieve satisfactory results. On the other hand, the Bayesian algorithm is expected to perform excellently in such situations. The poor outcomes are believed to be primarily due to the random selection of the starting state within the search space. This starting situation can lead to an initial state far from the optimal solution, thus requiring too many iterations to reach a feasible solution.

Both comparison algorithms have parameters that define their functionality. Each set of parameters was chosen to be different to prevent an incorrect parameter setting from negatively influencing all the experiments, with an effort to maintain appropriate values. However, the results were observed to vary minimally across different parameter sets. The strength of the proposed algorithms lies in starting from a known initial condition. Furthermore, the structure of these algorithms ensures that the parameters are only updated in

the case of improved execution, making it beneficial to have a more controlled exploration of the parameters in such situations.

Table III demonstrates how all the presented algorithms yield similar results, but some differences remain. Alg.2 exhibits a slower centering speed but achieves faster task completion. This result could be attributed to smaller parameter variations, which depend on the probability variable. Consequently, it results in slower centering due to working within a more comprehensive parameter range. At the same time, task completion is faster as it operates within a limited parameters workspace requiring slight variations. Alg.3 worsens both results, particularly in achieving the perfect task realization. This result could be due to the higher weight on the last execution without containing the parameter variation. Consequently, focusing on the correct solution becomes challenging because even slight variations in the vicinity can lead to significant differences in reward. Alg.4 brings slight improvements in both results. This result suggests that combining the two modifications to the algorithm can lead to enhancements. It is worth noticing that proper setting of the parameters a and b of Alg.4 may yield even better results. As Table III shows, the computational times of the algorithms are not used as quality indices because compared to the execution time of the task, they are negligible.

IV. Conclusions and future works

This article introduces a framework developed to aid in intuitive programming. It can optimize the parameters that define a task by employing dedicated optimization algorithms and a physical simulation environment. Simulations in a realistic environment show the proposed algorithm's effectiveness in optimizing the task parameter. These tests involve conducting optimization processes of a peg-in-hole task. The proposed algorithms have been compared against well-known algorithms, specifically Bayesian and genetic algorithms. The results highlight the effectiveness of the proposed method in reaching a successful solution in fewer iterations than the state-of-the-art algorithms. Moreover, the case study demonstrates that with a relatively modest number of optimization iterations (typically fewer than 200), it is possible to execute a task starting from initial infeasibility.

Future works will deal with further comparisons with additional algorithms, like particle swarm and support vector machine optimization. Additionally, research will be conducted into methods for executing tests in parallel, significantly reducing computational time. The sim-to-real transfer will also be studied to assess the framework's effectiveness in performing an initial optimization in a simulated environment, followed by refinement in physical application.

V. Ackhowledgement

This work is partially supported by the EcoCirc project (Lombardy, Italy, regional project, regional deliberation XI/4730 of the 17/05/2021) and by the "Intelligent Motion Control under Industry4.E" (IMOCO4.E) project, which has received funding from the ECSEL Joint Undertaking under grant agreement No. 101007311.

References

[1] M. Čech, A.-J. Beltman, and K. Ozols, "Digital twins and ai in smart motion control applications," in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2022, pp. 1–7.

[2] E. Coronado, F. Mastrogiovanni, B. Indurkhya, and G. Venture, "Visual programming environments for end-user development of intelligent and social robots, a systematic review," *Journal of Computer Languages*, vol. 58, p. 100970, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590118420300307

[3] D. Weintrop, "Block-based programming in computer science education," *Communications ACM*, vol. 62, pp. 22–25, 2019. [Online]. Available: https://cacm.acm.org/magazines/2019/8/238340-block-based-programming-in-computer-science-education/abstract

[4] M. Stenmark and P. Nugues, "Natural language programming of industrial robots," in *IEEE ISR 2013*, 2013, pp. 1–5.

[5] M. Petit and Y. Demiris, "Hierarchical action learning by instruction through interactive grounding of body parts and proto-actions," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 3375–3382.

[6] G. Apostolopoulos, D. Andronas, N. Fourtakas, and S. Makris, "Operator training framework for hybrid environments: An augmented reality module using machine learning object recognition," *Procedia CIRP*, vol. 106, pp. 102–107, 2022, 9th CIRP Conference on Assembly Technology and Systems. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827122001639

[7] D. Massa, M. Callegari, and C. Cristalli, "Manual guidance for industrial robot programming," *Ind. Robot*, vol. 42, pp. 457–465, 2015.

[8] P. N. Mohammad Safeea, "Precise positioning of collaborative robotic manipulators using hand-guiding," *The International Journal of Advanced Manufacturing Technology*, vol. 120, p. 5497–5508, 2022. [Online]. Available: https://doi.org/10.1007/s00170-022-09107-1

[9] S. Blankemeyer, R. Wiemann, L. Posniak, C. Pregizer, and A. Raatz, "Intuitive robot programming using augmented reality," *Procedia CIRP*, vol. 76, pp. 155–160, 2018, 7th CIRP Conference on Assembly Technologies and Systems (CATS 2018). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2212827118300933

[10] A. Angelidis and G. C. Vosniakos, "Prediction and compensation of relative position error along industrial robot end-effector paths," *International Journal of Precision Engineering and Manufacturing*, vol. 115, no. 1, pp. 63–73, 2014.

[11] V. Villani, F. Pini, F. Leali, and C. Secchi, "Survey on human–robot collaboration in industrial settings: Safety, intuitive interfaces and applications," *Mechatronics*, vol. 55, pp. 248–266, 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0957415818300321

[12] S. Makris, P. Tsarouchi, D. Surdilovic, and J. Krüger, "Intuitive dual arm robot programming for assembly operations," *CIRP Annals*, vol. 63, no. 1, pp. 13–16, 2014. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0007850614000201

[13] E. Villagrossi, N. Pedrocchi, and M. Beschi, "Simplify the robot programming through an action-and-skill manipulation framework," in *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2021, pp. 1–6.

[14] Stanford Artificial Intelligence Laboratory et al., "Robotic operating system," 2023. [Online]. Available: https://www.ros.org

[15] M. Faroni, A. Umbrico, M. Beschi, A. Orlandini, A. Cesta, and N. Pedrocchi, "Optimal task and motion planning and execution for multiagent systems in dynamic environments," *IEEE Transactions on Cybernetics*, pp. 1–12, 2023.

[16] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," http://pybullet.org, 2016–2023.

[17] A. F. Gad, "Pygad: An intuitive genetic algorithm python library," 2021.

[18] F. Nogueira, "Bayesian Optimization: Open source constrained global optimization tool for Python," 2014–2023. [Online]. Available: https://github.com/fmfn/BayesianOptimization